

PHASE – 4

Database Configuration

API Testing Demo :

1. Configure MongoDB:

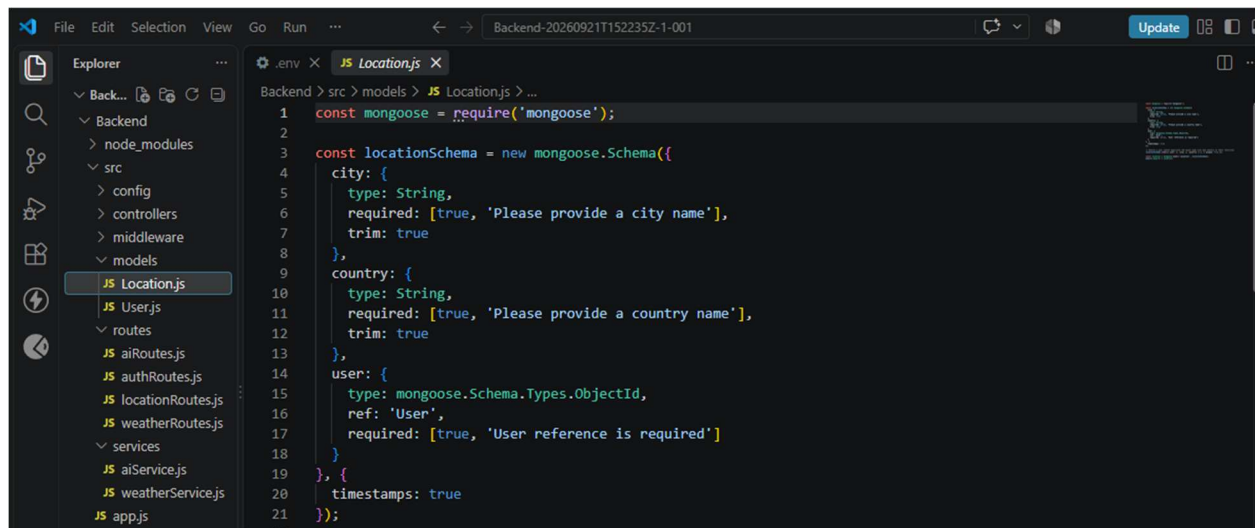
- Install Mongoose.
In your Node.js project directory, run:
`npm install mongoose`
- Create database connection.
- Create Schemas & Models.

Create Database connection:

Now, make sure the database is connected before performing any of the actions through the backend. The connection code looks similar to the one provided below

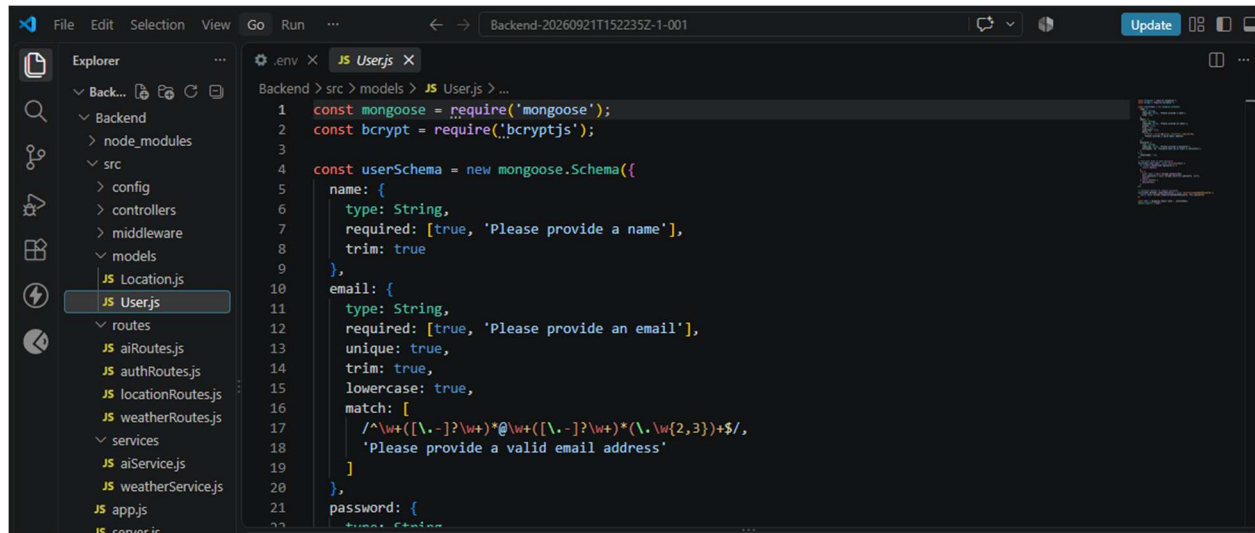
Mongoose Schema Definitions

Location Schema:

A screenshot of a Visual Studio Code editor window. The Explorer sidebar on the left shows a project structure with folders 'Backend' and 'src', and subfolders 'node_modules', 'config', 'controllers', 'middleware', 'models', 'routes', and 'services'. The 'models' folder is expanded, showing 'Location.js' selected. The main editor area displays the content of 'Location.js'. The code defines a Mongoose schema for a location, with fields for 'city' and 'country' (both strings, required, and trimmed) and a 'user' field (a reference to the 'User' model, required). It also includes timestamps. The code is as follows:

```
1 const mongoose = require('mongoose');
2
3 const locationSchema = new mongoose.Schema({
4   city: {
5     type: String,
6     required: [true, 'Please provide a city name'],
7     trim: true
8   },
9   country: {
10    type: String,
11    required: [true, 'Please provide a country name'],
12    trim: true
13  },
14   user: {
15     type: mongoose.Schema.Types.ObjectId,
16     ref: 'User',
17     required: [true, 'User reference is required']
18   }
19 }, {
20   timestamps: true
21 });
```

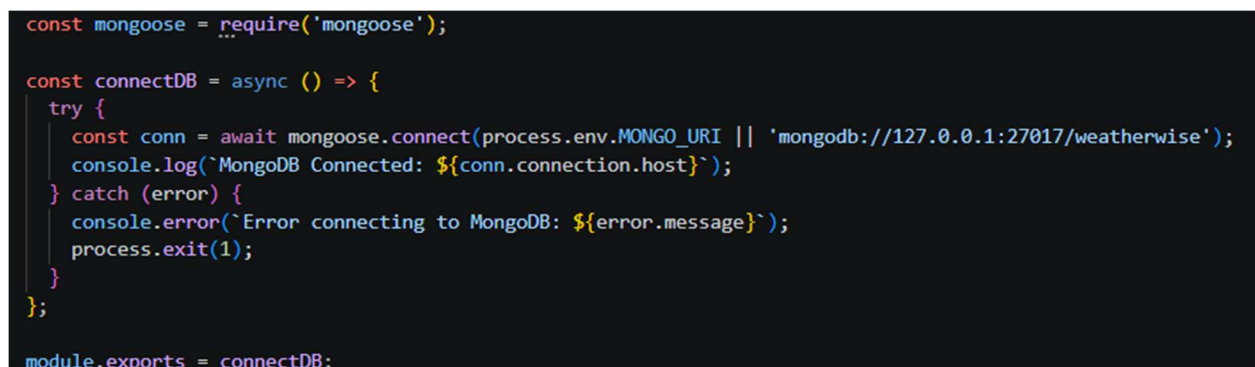
User schema

A screenshot of the Visual Studio Code editor. The Explorer sidebar on the left shows a project structure with folders 'Backend' and 'src'. Under 'src', there are folders 'config', 'controllers', 'middleware', 'models', 'routes', and 'services'. The 'models' folder is expanded, showing files 'Location.js' and 'User.js'. 'User.js' is selected and its content is displayed in the main editor. The code defines a Mongoose schema for a user with fields 'name', 'email', and 'password'.

```
1 const mongoose = require('mongoose');
2 const bcrypt = require('bcryptjs');
3
4 const userSchema = new mongoose.Schema({
5   name: {
6     type: String,
7     required: [true, 'Please provide a name'],
8     trim: true
9   },
10  email: {
11    type: String,
12    required: [true, 'Please provide an email'],
13    unique: true,
14    trim: true,
15    lowercase: true,
16    match: [
17      /^\\w+([\\.-]?\\w+)*@\\w+([\\.-]?\\w+)*(\\.\\w{2,3})+$/,
18      'Please provide a valid email address'
19    ]
20  },
21  password: {
22    type: String,
```

Db connection:

This module establishes a connection between the application and MongoDB using Mongoose. It ensures that the backend communicates with the database before processing any client requests.

A screenshot of a code editor showing a JavaScript function 'connectDB' that uses Mongoose to connect to a MongoDB database. The function is asynchronous and includes error handling. It sets the MONGO_URI environment variable to 'mongodb://127.0.0.1:27017/weatherwise'.

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/weatherwise');
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Error connecting to MongoDB: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```